

Exercices arbres binaires

1. Ecrire une version récursive d'une fonction qui calcule le nombre de feuilles d'un arbre binaire.
2. Ecrire une version récursive ainsi qu'une version itérative d'une fonction qui permet de vérifier si un arbre binaire est dégénéré.
3. Ecrire une fonction récursive qui permet de rechercher une valeur donnée dans un arbre binaire et qui retourne vrai si cette valeur a été trouvée et faux sinon.
4. Ecrire une procédure (récursive) qui, étant donné un arbre représenté d'une façon chaînée, permet de donner son équivalent en représentation contigue séquentielle.
5. Ecrire une procédure qui, étant donné un arbre représenté d'une façon contigue séquentielle, permet de donner son équivalent en représentation chaînée.

Correction

1-

Fonction Feuille (A : Arbre) : Entier

```
Var
Début
Si (A=Nil) alors
  Feuille ← 0
Sinon si (g(A)=Nil et d(A)=Nil) alors
  Feuille ← 1

  sinon
    Feuille ← Feuille(g(A)) + Feuille(d(A))
  Fsi
Fsi
Fin
```

2-

Version récursive

Fonction Dégénéré (A : Arbre) : Booléen

```
Var
Début
```

```

Si A=Nil alors
    Dégénéré ← vrai
Sinon si g(A) <> Nil et d(A) <> Nil alors
    Dégénéré ← faux
Sinon si g(A) = Nil alors
    Dégénéré ← Dégénéré (d(A))
Sinon
    Dégénéré ← Dégénéré (g(A))
Fsi
Fsi
Fsi
Fin

```

Version itérative

Fonction Dégénéré (A : Arbre) : Booléen

```

Var
Test : booléen
Début
Test ← vrai
Tant que (A <> Nil et Test) faire
    Si g(A) <> Nil et d(A) <> Nil alors
        Test ← faux
    Sinon si g(A) <> Nil alors
        A ← g(A)
    Sinon
        A ← d(A)
Fsi
Fsi
FTQ
Dégénéré ← Test
Fin

```

3-

Fonction Rechercher (A : Arbre, X : Element) : booléen

```

Var
Début
    Si A = Nil alors
        Rechercher ← faux
    Sinon Si (A^.val = X) alors
        Rechercher ← vrai
    Sinon si Rechercher(g(A)) alors

```

```

    Rechercher ← vrai
  Sinon
    Rechercher ← Rechercher(d(A))
  Fsi
Fsi
Fsi
Fin

```

4-

Procédure Arb-Tab(DON A : Arbre, i : entier, DONRES T : Tableau [1..Max] de Entier)

Ici on suppose qu'on traite un arbre d'entiers strictement positifs

```

Var
Début
  Si A= Nil alors
    Si i<=Max alors
      T[i] ← 0
    Fsi
  Sinon si i<=Max alors
    T[i] ← A^.val
    Arb-Tab(g(A),2i,T)
    Arb-Tab(d(A),2i+1,T)
  Sinon Ecrire ("dépassement de capacité")
  Fsi
Fsi
Fin

```

5-

Procédure Transforme (DON T : tableau [1..Max] de entier, i : entier, DONRES A : Arbre)

```

Var
Début
  Si T[i]=0 alors
    A ← Nil
  Sinon
    allouer(A)
    A^.val ← T[i]
  Si 2i<=Max alors
    Transforme(T,2i,A^.g)
  Sinon A^.g ← Nil
  Fsi
  Si 2i+1 <=Max alors

```

```

        Transforme(T,2i+1,A^.d))
    Sinon A^.d ← Nil
    Fsi
Fsi
Fin

Pile=struct
Elem :arbre
Indice :entier
Fin struct
PL : ^Pile
Procédure Arb-Tab( DON A : Arbre, i : entier, DONRES T : Tableau [1..Max] de Entier)
Var
q :arbre
PA : Pile
P :PL
Début
i ← 1
q ← A
Tant que (A <> Nil ou non Est-vide(P)) faire
    Tant que (A<> Nil ) faire
        T[i]← q^.val
        PA.indice ← i
        PA.Elem ← q
        Empiler(P,PA)
        q ← q^.g
        I ← 2*i
    FTQ
    Si i<=Max alors
        T[i] ← 0
    Fsi
    Depiler(P,PA)
    i ← 2*PA.indice + 1
    q ← PA.Elem
    q ← q^.d
FTQ
Fin

```